ENGR 102 Summer Bridge
Day 15 Student Workbook

Append, Concatenation, Slicing, and Strings as Sequences

Friday, July 24, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Process strings by index and build stored results with append, concatenation, and direct slice boundaries.

Allowed tools	Prior tools plus string indexing, in, direct slicing, list append, and list/string concatenation
Support supplied	The input/output contract and allowed sequence tools are supplied.
You must decide	Traversal state, collected-result state, slice boundaries, and validation order.
Scaffold level	Specification only; no planning prompts, variable inventory, or ordered algorithm.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	9	intermediate state, condition results, and exact output
Diagnosis and repair	7	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	34	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 9 points

Trace index selection, append state, slicing, and concatenation in one sequence ledger.

```
labels = ["A", "B"]
text = "loop"

for index in range(1, len(text), 2):
    labels.append(text[index])

tail = text[1:3]
combined = labels + [tail]
print(labels)
print(tail)
print(combined)
```

Your task

1. Give the complete index/list/slice state and all exact output lines. Explain why the slice stop is not included.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 7 points

The intent is to create [1, 2, 3], but the program fails before print.

```
values = [1, 2]
values = values + 3
print(values)
```

Your task

1. Name the actual failure, explain the type mismatch, give two minimal repairs with different sequence operations, and name one empty-list test.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 34 points across Pages 4-5

Program specification

- Read word as text. If its length is less than 4, print invalid and nothing else.
- Otherwise loop through every index and append characters at even indexes to even_characters.
- Count vowels using membership in the text aeiouAEIOU.
- Build edge_text by concatenating the first two-character slice and the last two-character slice.
- Print even_characters, vowel_count, and edge_text on separate lines.

Program workspace – begin the connected solution here.

Task 3 continued. Test and defend the program **included in 34 points**

Continue code if needed.

Required tests, expected results, and brief defense

VIP Lab Alignment

Bridge Day 15 • Contract c821cfcf7189

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 6 **Activities:** 18.23, 18.24

Apply filtering and the same normalization pipeline to strings before counting or comparing.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.23 `18-23-work` / `18-23-transfer`	**Count Kept Code Characters**	Traverse a string one character at a time.
18.24 `18-24-work` / `18-24-transfer`	**Badge ID Match**	Apply the same cleanup rule to both strings.

Readiness check before you code

1. For Activity 18.23, name the characters that do not count and the condition that skips them. _____

2. For Activity 18.24, write the cleanup operations in the same order for both identifiers. _____

Notebook and submission procedure

1. Work in the provided sample and transfer cells; do not delete or recreate them.
2. Run each cell and compare fresh output with the notebook's stated result before revising.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Contract-controlled page • generated from the current VIP activity profiles and stable notebook cell IDs